LydiaSyft: A Compositional Symbolic Synthesis Framework for LTL$_f$ Specifications

Shufang Zhu[1(✉)] and Marco Favorito[2]

[1] University of Liverpool, Liverpool, UK
shufang.zhu@liverpool.ac.uk
[2] Banca d'Italia, Rome, Italy
marco.favorito@bancaditalia.it

Abstract. There has been a massive interest in utilizing Linear Temporal Logic on finite traces (LTL$_f$) as a specification language in the last decade, particularly in reactive synthesis. This highlights the need for a unified and efficient framework to fulfil the increasing demand for easy-to-use implementations of synthesis (and reasoning) algorithms for LTL$_f$. To that end, we introduce LydiaSyft, an open-source compositional symbolic synthesis framework that integrates efficient data structures and techniques focused on LTL$_f$ specifications. LydiaSyft supports both explicit-DFA and symbolic-DFA construction from LTL$_f$ formulas, essential DFA manipulations, and offers an extensible framework for reactive synthesis of LTL$_f$ specifications, accommodating more complex synthesis scenarios. We demonstrate this feasibility by supporting LTL$_f$ synthesis as well as LTL$_f$ synthesis with LTL environment specifications expressed in various forms. LydiaSyft is highly efficient and versatile, providing user-friendly C++ interfaces and extensive benchmarks that cater to a diverse audience, including computer scientists, practitioners, students, and the reactive synthesis research community.

Keywords: Reactive synthesis · Linear Temporal Logic on finite traces · Symbolic synthesis · Open-source framework

1 Introduction

Recently, there has been massive interest in reactive synthesis for specifications expressed in Linear Temporal Logic on finite traces (LTL$_f$) [12]. The first LTL$_f$ synthesizer Syft demonstrated the practical scalability of LTL$_f$ synthesis [25], which opened the door to a whole dimension of utilizing LTL$_f$ in various domains such as planning [11], robotics [16], and reinforcement learning [6]. This massive interest in LTL$_f$ led to an increasing demand for easy-to-use implementations of synthesis and reasoning algorithms for LTL$_f$ specifications.

We introduce LydiaSyft, an open-source software framework for LTL$_f$ reasoning and synthesis. The reasoning component leverages LTL$_f$-to-DFA construction, reducing LTL$_f$ reasoning to automata-based reasoning. LydiaSyft adopts the

S. Zhu and M. Favorito—Equal contribution.

A. Gurfinkel and M. Heule (Eds.): TACAS 2025, LNCS 15696, pp. 295–302, 2025.
https://doi.org/10.1007/978-3-031-90643-5_15

compositional automata construction technique from Lydia [9], providing both explicit-state DFA and symbolic-state DFA construction and manipulation. Unlike Lydia and Syft, which handle only standard LTL_f synthesis (though Lydia introduces a novel compositional LTL_f-to-DFA construction technique), LydiaSyft supports a broader range of synthesis settings for LTL_f specifications, including LTL_f synthesis [13,25], maximally permissive strategy of LTL_f synthesis [24], LTL_f synthesis with simple Stability or Fairness environment specification [23], and LTL_f synthesis with Generalized-Reachability(1) environment specification [7,8]. LydiaSyft leverages the symbolic game-solving techniques from Syft [25] to deal with the synthesis problems and offers reusable C++ libraries of two-player game solvers. These solvers support customization, enabling users to create and efficiently solve custom games, e.g., the reachability game solver used in robotics task planning [16].

The primary goal of LydiaSyft is to provide an easy-to-extend platform for LTL_f reasoning and synthesis, focusing on usability and extensibility over peak performance optimizations, such as those for standard LTL_f synthesis [21,10,15,22]. Despite this focus, LydiaSyft features an efficient, modular C++ implementation of core data structures and synthesis techniques, earning it **2nd** place in the LTL_f synthesis track of SYNTCOMP 2024. Due to space constraints, additional experimental results and further technical details are provided in the Appendix.

LydiaSyft is available at https://github.com/whitemech/LydiaSyft, and the documentation at https://whitemech.github.io/LydiaSyft/index.html. The peer-reviewed artifact is available at https://zenodo.org/records/13927906.

2 Architecture

The main components of LydiaSyft's architecture are depicted in Figure 1 (All the techniques implemented in LydiaSyft also support Linear Dynamic Logic on finite traces (LDL_f) [12] specifications). LydiaSyft accepts LTL_f formulas written in a syntax following the TLSF format [18]. Depending on the synthesis task, LydiaSyft takes additional inputs capturing safety (expressed in LTL_f), simple Fairness, simple Stability, and Generalized-Reactivity(1) specifications [23,7,8].

Explicit-state DFA construction. LydiaSyft leverages the compositional LTL_f-to-DFA construction techniques from Lydia [9] to build an *explicit-state DFA* of the input LTL_f specification. This process involves a fully compositional technique that aggressively minimizes partial results so that the state space of the DFA is kept at the minimum. A novel contribution of our integration is to provide a better code interface to handle automata, enhancing interoperability with other modules and, potentially, with external tools.

Symbolic-state DFA construction. The constructed explicit-state DFA is then transformed into a *symbolic-state DFA*, both the state space and the transitions of which are represented symbolically in BDDs. This provides a compact form of the DFA, facilitating subsequent computational steps. In particular, the data structure used is known in the literature as *partitioned* DFA representation [25]. LydiaSyft provides novel implementations for representing and manipulating symbolic DFAs using the same interface used for explicit-state DFAs.

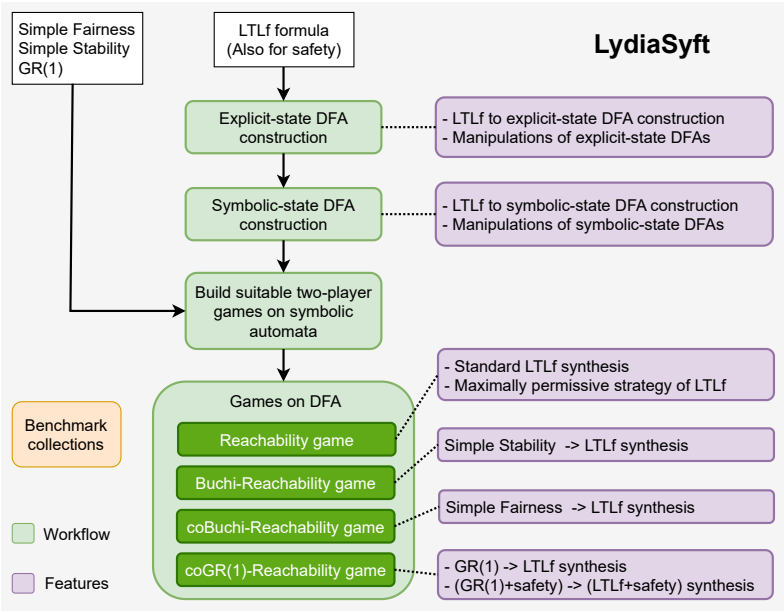

Fig. 1: An architecture overview of LydiaSyft.

Game construction. On top of the symbolic automata, LydiaSyft builds a two-player game data structure, possibly incorporating additional specifications like GR(1). These games form the backbone of the synthesis process, where the agent has to find a strategy that wins the game against the environment. LydiaSyft also supports adaptable player orders: agent-first or environment-first. We default to an agent-first setting in the rest of the paper, as in [13].

Games on DFA for synthesis. LydiaSyft supports various synthesis scenarios for LTL_f specifications through reducing to games on DFA, addressing diverse problems: (*i*) *reachability game*, for LTL_f synthesis [13,25] (*ii*) *maxset reachability game*, for computing the maximally permissive strategy of LTL_f synthesis [24]. (*iii*) *Büchi-Reachability game*, for LTL_f synthesis with simple Stability environment specifications [23]. (*iv*) *coBüchi-Reachability game*, for LTL_f synthesis with simple Fairness environment specifications [23]. (*vi*) *coGR(1)-Reachability game*, for LTL_f synthesis with GR(1) environment specifications, and possible additional safety conditions to both the environment and the agent [7,8]. A computed winning strategy, if it exists, can be printed in DOT format.

3 Compositional LTL_f-to-DFA Construction

The basic building block for LydiaSyft is the LTL_f-to-DFA construction. This section details how the explicit-state/symbolic-state DFA construction parts work and which data structures and procedures have been used.

LTL_f **Basics.** *Linear Temporal Logic on finite traces* (LTL_f) [12] is a specification language expressing temporal properties on finite, nonempty traces. In particular, LTL_f shares syntax with LTL, which is instead interpreted over infinite traces [20]. LydiaSyft also supports the variant that works for empty traces [4]. Given a set of atomic propositions $Prop$, LTL_f formulas are generated as follows: $\varphi ::= tt \mid a \mid \varphi \wedge \varphi \mid \neg\varphi \mid \bigcirc\varphi \mid \varphi \mathcal{U} \varphi$, where tt is the tautology, $a \in Prop$ is an *atom*, $\bigcirc$ (*Next*) and $\mathcal{U}$ (*Until*) are temporal operators. We make use of standard Boolean abbreviations, e.g., $ff \equiv \neg tt$, $\vee$ (or) and $\rightarrow$ (implies), *true* and *false*. In addition, we define the following abbreviations: *Weak Next* $\bullet\varphi \equiv \neg\bigcirc\neg\varphi$, *Eventually* $\Diamond\varphi \equiv true\,\mathcal{U}\,\varphi$ and *Always* $\Box\varphi \equiv false\,\mathcal{R}\,\varphi$, where $\mathcal{R}$ is for *Release*. The detailed semantics of LTL_f can be found in [12]. It is shown that, for every LTL_f formula φ, one can construct a Deterministic Finite Automaton (DFA) that accepts exactly the same language as the LTL_f formula [12]. Notably, an LTL_f formula can be transformed into an equivalent DFA in at most 2EXPTIME [12].

Build explicit-state DFA. An explicit-state DFA [17] is a tuple $\mathcal{A} = \langle \Sigma, S, s_0, \delta, F \rangle$, where: Σ is the alphabet, S is a finite set of states, $s_0 \in S$ is the initial state, $\delta : S \times \Sigma \rightarrow S$ is a total transition function and $F \subseteq S$ is the set of accepting states. Given an LTL_f formula φ, LydiaSyft leverages the compositional DFA construction in Lydia [9], which inductively applies a transformation procedure to each subformula of φ. This construction is "bottom-up", computing the DFAs of subformulas and combining them based on the LTL_f operator under transformation. LydiaSyft extends Lydia's DFA library, which represents explicit-state DFAs using *Shared Multi-terminal Binary Decision Diagrams* (ShMTBDDs) [5,19]. To ensure compatibility with subsequent symbolic-state DFA construction, LydiaSyft provides convenient wrappers for these libraries. Notably, LydiaSyft enhances explicit-state DFA manipulation by offering operations such as complement, product, state and transition restrictions, and printing.

Build symbolic-state DFA. Given an explicit-state DFA $\mathcal{A} = (2^{\mathcal{X} \cup \mathcal{Y}}, S, s_0, \delta, F)$, the corresponding symbolic-state DFA is defined as $\mathcal{D} = (\mathcal{X}, \mathcal{Y}, \mathcal{Z}, \iota, \eta, f)$, where $\mathcal{X}$ and $\mathcal{Y}$ are as in $\mathcal{A}$, and $\mathcal{Z}$ is a set of $\lceil \log_2 |S| \rceil$ propositions, with each state $s \in S$ corresponding to an interpretation $Z \in 2^{\mathcal{Z}}$. The initial state s_0 is represented by $\iota \in 2^{\mathcal{Z}}$. The transition function $\eta = \{\eta_0, \eta_1, \cdots, \eta_k\}$ ($|\eta| = |\mathcal{Z}|$) maps interpretations $X \in 2^{\mathcal{X}}$, $Y \in 2^{\mathcal{Y}}$ and $Z \in 2^{\mathcal{Z}}$ to the successor state $\delta(s, X \cup Y)$ such that $\eta_i(X, Y, Z) = 1$ iff the value of $z_i \in \mathcal{Z}$ is 1 in the interpretation $Z' \in 2^{\mathcal{Z}}$ corresponding to $\delta(s, X \cup Y)$. Finally, the set of accepting states f is a Boolean formula over $\mathcal{Z}$, satisfied by Z iff it corresponds to an accepting state $s \in F$ [25]. LydiaSyft utilizes the same symbolic-state DFA representation as Syft, which represents a symbolic-state DFA $\mathcal{D}$ as a sequence of BDDs. However, while Syft manually converts explicit-state DFAs (represented as ShMTBDDs) into symbolic-state DFAs by traversing the ShMTBDD and generating propositional evaluations for each explicit state, LydiaSyft takes a different approach. It employs Algebraic Decision Diagrams (ADDs) [2] as an intermediate step, leveraging the BDD/ADD library CUDD-3.0.0. We first convert a ShMTBDD to a sequence of ADDs, and then use the rich API functions, in particular BddIthBit, provided by CUDD to obtain the BDD sequence, avoiding the manual conversion.

Additionally, LydiaSyft supports a range of symbolic-state DFA manipulations, including complement, product, state and transition restrictions, and printing.

4 Synthesis Settings for LTL_f Specifications

This section outlines the synthesis settings for LTL_f specifications supported by LydiaSyft. Let $\mathcal{X}$ and $\mathcal{Y}$ be Boolean variables, with $\mathcal{X}$ controlled by the environment and $\mathcal{Y}$ controlled by the agent. An *agent strategy* is a function $\sigma_{agn} : (2^{\mathcal{X}})^* \rightarrow 2^{\mathcal{Y}}$, and an environment strategy is a function $\sigma_{env} : (2^{\mathcal{Y}})^+ \rightarrow 2^{\mathcal{X}}$. An agent strategy *induces* a trace $\pi = (X_0 \cup Y_0)(X_1 \cup Y_1)\cdots \in (2^{\mathcal{X}\cup\mathcal{Y}})^\omega$ if $\sigma_{agn}(\epsilon) = Y_0$ and $\sigma_{agn}(X_0X_1\cdots X_j) = Y_{j+1}$ for every $j \geq 0$. An environment strategy *induces* a trace $\pi \in (2^{\mathcal{X}\cup\mathcal{Y}})^\omega$ if $\sigma_{env}(Y_0Y_1\cdots Y_j) = X_j$ for every $j \geq 0$. The unique trace induced by an agent strategy σ_{agn} and an environment strategy σ_{env} is denoted by $\mathsf{play}(\sigma_{agn}, \sigma_{env})$ or simply π when clear from the context.

LTL_f Synthesis. The problem is described as a tuple $\mathcal{P} = (\mathcal{X}, \mathcal{Y}, \varphi)$, where φ is an LTL_f formula over $\mathcal{X}\cup\mathcal{Y}$. Realizability of $\mathcal{P}$ checks whether $\exists\sigma_{agn}\forall\sigma_{env} : \exists k \geq 0.\mathsf{play}^k(\sigma_{agn}, \sigma_{env}) \models \varphi$. Synthesis of $\mathcal{P}$ computes a strategy σ_{agn} if exists [13]. LydiaSyft integrates the preprocessing techniques for LTL_f synthesis introduced in [21] for effective *realizability check* and *unrealizability check*. If preprocessing fails, LydiaSyft constructs a symbolic-state DFA $\mathcal{D}_\varphi$ for φ, and builds a reachability game $G = (\mathcal{D}_\varphi, \texttt{Reach}(t))$, where t is a set of target states. LydiaSyft follows the symbolic synthesis technique presented in [25] to solve $G = (\mathcal{D}_\varphi, \texttt{Reach}(t))$, providing a C++ library Reachability.h.

LTL_f MaxSet Synthesis. LTL_f maxset synthesis $\mathcal{P} = (\mathcal{X}, \mathcal{Y}, \varphi)$ aims at computing the maximally permissive strategy (MaxSet), which represents the set of all agent winning strategies [24]. As shown in [24], MaxSet for $\mathcal{P} = (\mathcal{X}, \mathcal{Y}, \varphi)$ can be solved by reasoning about the maxset reachability game on $\mathcal{D}_\varphi$. LydiaSyft provides a C++ library ReachabilityMaxSet.h, which takes the reachability game $G = (\mathcal{D}_\varphi, \texttt{Reach}(t))$ as input and computes two nondeterministic strategies characterizing MaxSet, if there is at least one winning strategy.

LTL_f Synthesis with Simple Stability/Fairness Env. Specs. We now outline the LydiaSyft-supported synthesis settings for LTL_f specifications involving two-player games on DFAs that extend beyond reachability, focusing specifically on LTL_f synthesis with environment specifications. Following [1], we specify the environment behaviour by an LTL formula *env* and call it *environment specification*, and the agent goal is an LTL_f formula φ. Formally, given an LTL formula *env*, we say that an environment strategy *enforces env*, written $\sigma_{env} \rhd env$, if for every agent strategy σ_{agn} we have $\mathsf{play}(\sigma_{agn}, \sigma_{env}) \models env$. The problem of LTL_f *synthesis with environment specifications* is to find an agent strategy σ_{agn} such that $\exists\sigma_{agn}\forall\sigma_{env} \rhd env : \exists k.\mathsf{play}^k(\sigma_{agn}, \sigma_{env}) \models \varphi$.

An LTL formula *env* is considered as a simple Stability specification if it is of the form $\Diamond\Box(\alpha)$, and a simple Fairness specification if it is of the form $\Box\Diamond(\alpha)$, where α is a Boolean formula involving only environment variables [23]. As shown in [23], LTL_f synthesis with simple Stability/Fairness environment specification can be reduced to Büchi-Reachability and coBüchi-Reachability

games on $\mathcal{D}_\varphi$, respectively. LydiaSyft provides a C++ library BuchiReachability.h, which takes a constructed Büchi-Reachability game $G = (\mathcal{D}_\varphi, \texttt{BuchiReach}(t, \beta))$ as input, performs a nested fixpoint computation to solve the game. The library coBuchiReachability.h supports coBüchi-Reachability game solving.

LTL_f Synthesis with Generalized Reachability(1) Env. Specs. GR(1) is a syntactic fragment of LTL, providing a powerful notion of fairness [3]. The problem of LTL_f synthesis with GR(1) environment specifications introduced in [7,8] successfully brought the advances of GR(1) synthesis and LTL_f synthesis. LydiaSyft employs the solution proposed in [7,8] reducing the synthesis problem to a coGR(1)-Reachability game. LydiaSyft also allows adding additional safety conditions to both the environment and the agent, expressed in LTL_f formulas, as in [7,8]. It provides a C++ library coGR1Reachability.h, which takes a coGR(1)-Reachability game (with possible safety conditions) as input, reduces it further to a GR(1) game and uses Slugs [14] to solve it.

An example of using LydiaSyft as a CLI tool for LTL_f synthesis is shown in Listing 1.1.

Listing 1.1: Example CLI Commands for LTL_f Synthesis

```
#Usage:
> LydiaSyft synthesis --help
solve a classical LTLf synthesis problem
Usage: LydiaSyft synthesis [OPTIONS]

Options:
  -h,--help                   Print this help message and exit
  --help-all                  Expand all help
  -f,--spec-file TEXT:FILE REQUIRED
                              Specification file
  -s,--syfco-path TEXT:FILE   Path to Syfco binary
#Example of usage:
> LydiaSyft synthesis -f examples/test.tlsf   # UNREALIZABLE
> LydiaSyft synthesis -f examples/test1.tlsf  # REALIZABLE
```

5 Conclusion and Future Works

In this paper, we presented LydiaSyft, an open-source synthesis framework that integrates efficient data structure and synthesis techniques, focusing on agent goals expressed in LTL_f specifications. We believe that LydiaSyft provides a significant foundation to address the increasing demand for synthesis and reasoning of LTL_f specifications. LydiaSyft offers researchers, practitioners, and students convenient access to flexible automata construction from LTL_f specifications, and various synthesis settings. Furthermore, it provides a unified and accessible approach for reproducibility. We are actively working on the framework to integrate further and future synthesis settings into LydiaSyft. Using the permissive MIT license, we hope that LydiaSyft will be used and extended by various communities that are closely connected to LTL_f synthesis, such as reactive synthesis, AI planning, and task planning in robotics.

References

1. Aminof, B., De Giacomo, G., Murano, A., Rubin, S.: Planning under LTL environment specifications. In: ICAPS. pp. 31–39 (2019)
2. Bahar, R., Frohm, E., Gaona, C., Hachtel, G., Macii, E., Pardo, A., Somenzi, F.: Algebraic decision diagrams and their applications. In: ICCAD. pp. 188–191 (1993)
3. Bloem, R., Jobstmann, B., Piterman, N., Pnueli, A., Sa'ar, Y.: Synthesis of Reactive(1) designs. J. Comput. Syst. Sci. **78**(3), 911–938 (2012)
4. Brafman, R.I., De Giacomo, G., Patrizi, F.: $\mathrm{LTL}_f/\mathrm{LDL}_f$ non-markovian rewards. In: AAAI. pp. 1771–1778 (2018)
5. Bryant, R.E.: Symbolic Boolean Manipulation with Ordered Binary-Decision Diagrams. ACM Comput. Surv. **24**(3) (1992)
6. Camacho, A., Icarte, R.T., Klassen, T.Q., Valenzano, R.A., McIlraith, S.A.: LTL and beyond: Formal languages for reward function specification in reinforcement learning. In: IJCAI. pp. 6065–6073 (2019)
7. De Giacomo, G., Di Stasio, A., M. Tabajara, L., Y. Vardi, M., Zhu, S.: Finite-trace and generalized-reactivity specifications in temporal synthesis. In: IJCAI. pp. 1852–1858 (2021)
8. De Giacomo, G., Di Stasio, A., Tabajara, L.M., Vardi, M.Y., Zhu, S.: Finite-trace and generalized-reactivity specifications in temporal synthesis. In: Formal Methods Syst. Des. (2023)
9. De Giacomo, G., Favorito, M.: Compositional approach to translate $\mathrm{LTL}_f/\mathrm{LDL}_f$ into deterministic finite automata. In: ICAPS. pp. 122–130 (2021)
10. De Giacomo, G., Favorito, M., Li, J., Vardi, M.Y., Xiao, S., Zhu, S.: Ltl_f synthesis as AND-OR graph search: Knowledge compilation at work. In: IJCAI. pp. 2591–2598 (2022)
11. De Giacomo, G., Fuggitti, F.: FOND4LTL: FOND Planning for LTL/PLTL Goals as a Service (2021)
12. De Giacomo, G., Vardi, M.Y.: Linear Temporal Logic and Linear Dynamic Logic on Finite Traces. In: IJCAI. pp. 854–860 (2013)
13. De Giacomo, G., Vardi, M.Y.: Synthesis for LTL and LDL on Finite Traces. In: IJCAI. pp. 1558–1564 (2015)
14. Ehlers, R., Raman, V.: Slugs: Extensible GR(1) synthesis. In: CAV. Lecture Notes in Computer Science, vol. 9780, pp. 333–339 (2016)
15. Favorito, M.: Forward LTL_f synthesis: DPLL at work. In: IPS-RCRA-SPIRIT@AIxIA. vol. 3585 (2023)
16. He, K., Wells, A.M., Kavraki, L.E., Vardi, M.Y.: Efficient Symbolic Reactive Synthesis for Finite-Horizon Tasks. In: ICRA. pp. 8993–8999 (2019)
17. Hopcroft, J.E., Motwani, R., Ullman, J.D.: Introduction to automata theory, languages, and computation, 3rd Edition. Pearson international edition, Addison-Wesley (2007)
18. Jacobs, S., Perez, G.A., Schlehuber-Caissier, P.: The temporal logic synthesis format tlsf v1.2 (2023)
19. Morten, B., Nils, K., Theis, R.: Mona: decidable arithmetic in practice (demo). In: FTRTFT. pp. 459–462 (1996)
20. Pnueli, A.: The temporal logic of programs. In: FOCS. pp. 46–57 (1977)
21. Xiao, S., Li, J., Zhu, S., Shi, Y., Pu, G., Y. Vardi, M.: On-the-fly synthesis for LTL over finite traces. In: AAAI. pp. 6530–6537 (2021)
22. Xiao, S., Li, Y., Huang, X., Xu, Y., Li, J., Pu, G., Strichman, O., Vardi, M.Y.: Model-guided synthesis for LTL over finite traces. In: VMCAI. pp. 186–207 (2024)

23. Zhu, S., Giacomo, G.D., Pu, G., Vardi, M.Y.: LTL$_f$ synthesis with fairness and stability assumptions. In: AAAI. pp. 3088–3095 (2020)
24. Zhu, S., De Giacomo, G.: Synthesis of maximally permissive strategies for LTL$_f$ specifications. In: IJCAI. pp. 2783–2789 (2022)
25. Zhu, S., Tabajara, L.M., Li, J., Pu, G., Vardi, M.Y.: Symbolic LTL$_f$ Synthesis. In: IJCAI. pp. 1362–1369 (2017)